

PLAN DE FORMATION — 3 SEMAINES

Création d'un Quiz interactif en HTML / CSS / JavaScript

Basé sur l'analyse et la reconstruction pas-à-pas du fichier quiz-corrige.html

Objectifs pédagogiques

- Comprendre et savoir reconstruire un mini-projet web complet (HTML + CSS + JavaScript vanilla, sans framework).
- Maîtriser la manipulation du DOM : sélection, création dynamique d'éléments, gestion d'événements.
- Savoir modéliser des données (tableau d'objets) et les relier à une interface utilisateur.
- Gérer un état applicatif simple (score, index courant, verrouillage) et son cycle de vie.
- Être capable d'étendre un projet existant avec de nouvelles fonctionnalités de façon autonome.

Vue d'ensemble du planning

Jour	Durée / Thème	Objectifs & contenu détaillé	Exercice pratique / Livrable
S1	Semaine 1	HTML & CSS — structure statique et habillage visuel du quiz (carte, barre de progression, boutons).	Livrable : version statique complète, sans JavaScript.
S2	Semaine 2	JavaScript — modélisation des données, DOM, logique de jeu, affichage dynamique des questions.	Livrable : quiz jouable de bout en bout (3 questions).
S3	Semaine 3	Finalisation — écran de résultat, rejouabilité, accessibilité, fonctionnalités bonus, documentation.	Livrable : quiz final complet + README.

Rythme suggéré : 5 séances par semaine (du lundi au vendredi), 2 à 3 heures par séance, soit environ 35 heures de formation au total.

SEMAINE 1 — Fondations HTML & CSS

Objectif de la semaine : obtenir une reproduction visuelle fidèle du quiz (carte centrée, barre de progression, boutons stylés), entièrement statique, avant d'y ajouter la moindre logique.

Jour	Durée / Thème	Objectifs & contenu détaillé	Exercice pratique / Livrable
J1	2h Structure HTML & sémantique	Analyser la structure du quiz fourni : <div id="carte">, #ecran-quiz, #resultat. Comprendre la hiérarchie : compteur, barre de progression, question, liste de réponses. Créer son propre squelette HTML vide reprenant ces mêmes identifiants (id). Discuter pourquoi on utilise des id uniques plutôt que des class pour cibler des éléments en JS.	Recréer à la main (sans copier) le fichier HTML de base avec les 7 zones : carte, compteur, barre-progres, barre-remplie, question, liste-reponses, resultat. Livrable : page HTML vide qui s'affiche sans erreur console.
J2	2h	Flexbox sur body : align-items, justify-	Styliser la carte pour qu'elle soit

Jour	Durée / Thème	Objectifs & contenu détaillé	Exercice pratique / Livrable
	CSS — mise en page de la carte	content pour centrer #carte verticalement/horizontalement. Box model : padding, border-radius, box-shadow sur #carte. Palette de couleurs sombres (#1a1f2e, #242b3d, #2f3850) et rôle de chaque teinte.	centrée, avec ombre portée et coins arrondis. Livrable : capture d'écran comparée à l'original.
J3	2h Barre de progression & transitions CSS	Construire #barre-progres (conteneur) + #barre-remplie (barre animée). Propriété transition:width .3s pour une animation fluide lors du changement de largeur. Lien entre une valeur CSS (width en %) et une donnée JS (index de question).	Faire varier manuellement le style.width de la barre via la console du navigateur et observer l'animation. Livrable : barre de progression fonctionnelle et animée.
J4	2,5h Boutons de réponse & états visuels	Classe .reponse : apparence de base, hover, transition .15s. États dynamiques ajoutés en JS : .correcte (vert #4ade80/#1f4a34) et .incorrecte (rouge #f87171/#4a1f24). Pseudo-état :disabled et son style (cursor:default) pour empêcher un second clic.	Créer 4 boutons de réponse statiques et simuler à la souris les 3 états (normal, correcte, incorrecte) en ajoutant/retirant des classes dans l'inspecteur. Livrable : feuille de style complète pour les boutons.
J5	1,5h Révision & responsive	Revoir max-width:90vw pour l'adaptation mobile. Vérifier le rendu sur plusieurs largeurs d'écran (DevTools responsive mode). Consolider le vocabulaire CSS vu (flex, transition, box-shadow, pseudo-classes).	Quiz de révision oral/écrit sur les propriétés CSS utilisées. Livrable : version HTML/CSS statique complète du quiz (sans interactivité), validée sur mobile et desktop.

SEMAINE 2 — Logique JavaScript & interactivité

Objectif de la semaine : donner vie à la page statique en la connectant à un tableau de données et en implémentant le cycle complet d'une question (affichage → clic → correction → question suivante).

Jour	Durée / Thème	Objectifs & contenu détaillé	Exercice pratique / Livrable
J6	2h Modéliser les données : tableaux d'objets	Étudier le tableau QUESTIONS : chaque question = objet {texte, reponses[], correcte}. Comprendre pourquoi correcte est un index (0,1,2...) et non le texte de la réponse. Bonnes pratiques : commentaires expliquant la donnée, cohérence des tableaux de réponses.	Écrire son propre tableau de 3 questions sur un thème choisi, avec 4 réponses chacune. Livrable : fichier JS contenant uniquement le tableau QUESTIONS personnalisé.
J7	2h Sélection du DOM	document.getElementById pour chaque élément (elQuestion,	Relier chaque variable elXxx à son id HTML correspondant et vérifier dans la

Jour	Durée / Thème	Objectifs & contenu détaillé	Exercice pratique / Livrable
		elListeReponses, elCompteur...). Différence entre stocker une référence dans une constante et la rechercher à chaque fois. Utiliser innerHTML = "" pour vider un conteneur avant de le remplir dynamiquement.	console (console.log). Livrable : liste commentée des références DOM utilisées.
J8	2,5h Fonctions & génération dynamique	Étudier afficherQuestion() : mise à jour texte, compteur, largeur de la barre. forEach sur q.reponses pour créer un <button> par réponse (document.createElement). Attacher un gestionnaire d'événement onclick avec une fonction fléchée capturant l'index i.	Implémenter afficherQuestion() à partir de zéro pour son propre tableau QUESTIONS. Livrable : la première question s'affiche automatiquement avec ses 4 boutons cliquables.
J9	2,5h Gestion de l'état & logique de validation	Variables d'état globales : indexActuel, score, verrouille — leur rôle et pourquoi il faut les réinitialiser. Fonction validerReponse(indexClique, boutonClique) : comparaison avec q.correcte, incrémentation du score. Pourquoi verrouille empêche les clics multiples (garde au tout début de la fonction). Désactivation de tous les boutons via querySelectorAll + disabled = true.	Implémenter validerReponse : colorier la bonne/mauvaise réponse et désactiver les boutons. Livrable : on peut répondre à une question et voir le retour visuel immédiat.
J10	2h Enchaînement asynchrone avec setTimeout	Rôle de setTimeout(passerASuivant, 900) : laisser le temps de voir la correction avant de changer de question. Fonction passerASuivant() : incrémenter indexActuel puis choisir entre afficherQuestion() et afficherResultat(). Discussion sur les callbacks et la nature non bloquante de setTimeout.	Châîner les questions : après un clic, attendre ~1s puis passer à la suivante automatiquement. Livrable : parcours complet des 3 questions sans rechargement de page.

SEMAINE 3 — Finalisation, robustesse et fonctionnalités avancées

Objectif de la semaine : boucler le cycle de vie complet du quiz (résultat, rejouer), améliorer la qualité (accessibilité, tests) et aller au-delà du fichier d'origine avec au moins une fonctionnalité bonus.

Jour	Durée / Thème	Objectifs & contenu détaillé	Exercice pratique / Livrable
J11	2h Écran de résultat	Basculer l'affichage : elEcranQuiz.style.display='none' / elResultat.style.display='block'. Calcul du pourcentage : Math.round((score/QUESTIONS.length)*100).	Implémenter afficherResultat() avec au moins 3 messages différents selon le pourcentage. Livrable : écran de résultat complet et lisible.

Jour	Durée / Thème	Objectifs & contenu détaillé	Exercice pratique / Livrable
		Messages conditionnels selon le score (if/else if/else) — personnaliser les seuils et les textes.	
J12	2h Rejouer & réinitialisation d'état	Fonction recommencer() : remise à zéro de indexActuel et score, ré-affichage de l'écran quiz. Piège fréquent : oublier de réinitialiser une variable et garder l'ancien score. Attacher l'écouteur au bouton #rejouer une seule fois (hors des fonctions répétées).	Implémenter le bouton Rejouer et vérifier qu'un second passage donne un score correct. Livrable : le quiz peut être rejoué indéfiniment sans bug d'état.
J13	2,5h Accessibilité & robustesse	Ajouter des attributs ARIA (aria-live sur le compteur/résultat) pour les lecteurs d'écran. Contraste des couleurs correcte/incorrecte (vérifier WCAG AA). Gestion d'erreurs simples : que se passe-t-il si QUESTIONS est vide ? Ajouter une vérification.	Auditer le quiz avec les outils d'accessibilité du navigateur et corriger au moins 2 problèmes. Livrable : rapport d'audit + correctifs appliqués.
J14	3h Fonctionnalités avancées (au choix)	Chronomètre par question (setInterval + affichage du temps restant). Sauvegarde du meilleur score avec localStorage. Mélange aléatoire des questions et des réponses (Fisher-Yates shuffle). Import du tableau QUESTIONS depuis un fichier JSON externe (fetch).	Choisir 1 à 2 fonctionnalités et les implémenter proprement, sans casser l'existant. Livrable : quiz enrichi avec au moins une fonctionnalité bonus démontrable.
J15	2h Tests, nettoyage & documentation finale	Relecture complète du code : cohérence des noms de variables, commentaires à jour. Tests manuels : parcours complet, cas limites (clic rapide, redimensionnement fenêtre). Rédiger un court README expliquant la structure du fichier et comment ajouter des questions.	Test croisé en binôme : faire tester son quiz par un pair et noter les retours. Livrable final : fichier HTML unique fonctionnel + README.md de présentation.

Grille d'auto-évaluation finale

À la fin des 3 semaines, l'apprenant doit être capable de cocher chacun des points suivants :

- Je peux expliquer, ligne par ligne, ce que fait chaque fonction du fichier quiz-corrigé.html.
- Je peux ajouter une nouvelle question au tableau QUESTIONS sans rien casser.
- Je peux expliquer la différence entre l'index d'un tableau et une valeur qu'il contient (cas de correcte).
- Je peux décrire le rôle de la variable verrouille et pourquoi elle est nécessaire.
- Je peux styliser un état interactif (hover, disabled, classe ajoutée dynamiquement) en CSS.
- Je sais utiliser setTimeout pour séquencer une action dans le temps.

- J'ai implémenté au moins une fonctionnalité qui n'existait pas dans le fichier original.

Ressources complémentaires

- MDN Web Docs — référence HTML, CSS, JavaScript (developer.mozilla.org).
- DevTools du navigateur — onglets Éléments, Console, Accessibilité, Mode responsive.
- Valideur W3C pour vérifier la conformité du HTML produit.